

DeepSeek-V3.2 论文详细分析报告

论文标题: DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

机构: DeepSeek-AI

发布日期: 2026年 (基于 DeepSeek-V3.1-Terminus 继续训练)

目录

- [概述与核心贡献](#)
- [DeepSeek Sparse Attention \(DSA\)](#)
- [后训练: 扩展 RL 与 Agent 合成](#)
- [评测结果](#)
- [工程实践亮点](#)
- [局限性与未来方向](#)

1. 概述与核心贡献

1.1 研究动机

在 DeepSeek-R1 发布后, 开源社区虽持续进步, 但闭源模型 (如 Anthropic Claude、Google Gemini) 的性能加速更快, 开源与闭源的差距反而在扩大。DeepSeek-V3.2 旨在通过三大突破解决开源模型在复杂任务中的三项关键缺陷:

- 架构效率:** 传统注意力在长序列上效率极低, 阻碍可扩展部署和有效后训练
- 资源分配:** 开源模型后训练阶段计算投入不足, 限制困难任务性能
- Agent 泛化:** 开源模型在工具使用场景中泛化和指令遵循能力明显落后

1.2 三大核心创新

创新	说明
DeepSeek Sparse Attention (DSA)	高效稀疏注意力, 将核心注意力复杂度从 $O(L^2)$ 降至 $O(Lk)$ ($k \ll L$)
可扩展 RL 框架 (Scalable GRPO)	后训练计算预算超过预训练成本的 10%, 多项稳定训练技术
大规模 Agent 任务合成流水线	1800+ 合成环境、85000+ 复杂 prompt, 驱动 RL 提升泛化能力

1.3 模型定位

DeepSeek-V3.2 是从 DeepSeek-V3.1-Terminus 基座检查点出发，通过**继续预训练 + 后训练**得到。存在两个变体：

变体	训练数据	长度惩罚	定位
DeepSeek-V3.2	推理 + Agent + 人类对齐	标准	通用高性能（成本效率优先）
DeepSeek-V3.2-Speciale	仅推理	降低	极致推理（IMO/IOI 金牌）

1.4 核心性能对标

1	基准测试对比（Thinking模式）：			
2		DS-V3.2	GPT-5-High	Gemini-3.0-Pro
3	AIME 2025	93.1	94.6	95.0
4	HMMT Feb 2025	92.5	88.3	97.5
5	HLE (text)	25.1	26.3	37.7
6	Codeforces Rating	2386	2537	2708
7	SWE Verified	73.1	74.9	76.2
8	Terminal Bench 2.0	46.4	35.2	54.2
9	Tool-Decathlon	35.2	29.0	36.4

2. DeepSeek Sparse Attention (DSA)

这是 V3.2 唯一的架构变更——除此之外所有模型配置、超参数与 V3.1-Terminus 完全相同。DSA 是 V3.2 效率突破的核心，将注意力计算从"每查询看所有 token"变为"每查询看精选的 k 个 token"。

2.1 DSA 设计原理与数学推导

DSA 由两个核心组件构成：Lightning Indexer（决定该看哪些 token）和细粒度 Token 选择器（执行稀疏注意力）。

2.1.1 Lightning Indexer：全序列扫描的轻量替代

为查询 token $h_t \in \mathbb{R}^d$ 和每个前序 token $h_s \in \mathbb{R}^d$ 计算索引分数：

$$\mathbf{I}_{t,s} = \sum_{j=1}^{H^I} w_t^{I,j} \cdot \text{ReLU} \left(q_t^{I,j} \cdot k_s^I \right)$$

其中各项的来源和维度：

- 索引器 Query: $q_t^{I,j} \in \mathbb{R}^{d^I}$ 从 h_t 通过低秩投影生成
- 索引器 Key: $k_s^I \in \mathbb{R}^{d^I}$ 从 h_s 通过低秩投影生成（与 query 投影不同，独立参数矩阵）
- 索引头权重: $w_t^{I,j} \in \mathbb{R}$ 从 h_t 通过另一个可学习矩阵投影得到——这使得不同的索引头对

最终分数的贡献度可以自适应调节

- H^I 为索引器头数——远小于主注意力的查询头数
- d^I 为索引器维度——同样小于主注意力维度

为什么选择 ReLU 而非 Softmax?

这是一个经过权衡的工程决策：

1. **FP8 效率：**ReLU 仅需逐元素比较——在现代 GPU 上 FP8 ReLU 比 FP8 exp (Softmax 所需) 快约一个数量级
2. **稀疏性：**ReLU 天然产生稀疏的索引分数——负相关的 KV 对直接归零，这恰好符合"只保留最重要的 KV 对"的设计理念
3. **梯度稳定性：**与 Softmax 相比，ReLU 在正值区域梯度为常数 1，避免了指运算带来的梯度消失/爆炸

与标准注意力的本质区别：

标准注意力中，softmax 使得所有 token 的注意力权重之和为 1（关注度守恒）。DSA 的索引分数没有这种归一化约束——一个 token 的索引总分可以是任意正值，仅反映它"可能被关注"的绝对强度，而非概率分布。

2.1.2 细粒度 Token 选择与稀疏注意力

基于索引分数，对每个查询 token h_t 做 Top-k 选择 ($k = 2048$)，仅保留分数最高的 k 个前序 token 的 KV 项参与核心注意力：

$$\mathbf{u}_t = \text{Attn}(h_t, \{c_s \mid \mathbf{I}_{t,s} \in \text{Top-}k(\mathbf{I}_{t,:})\})$$

这里 c_s 是第 s 个 token 的 KV 项——在 MLA 中是被 MQA 模式共享的压缩隐向量。

复杂度全景图：

组件	注意力操作	复杂度	在总计算中的占比
Lightning Indexer QK 内积	$q_t^I \cdot k_s^I$ (所有对)	$O(L^2 \cdot H^I \cdot d^I)$	极低： H^I 和 d^I 远小于主注意力
主模型核心注意力	仅选中的 k 对	$O(L \cdot k \cdot H \cdot d)$	大幅降低： $k = 2048 \ll L$
标准密集注意力	所有 L 对	$O(L^2 \cdot H \cdot d)$	100% (基线)

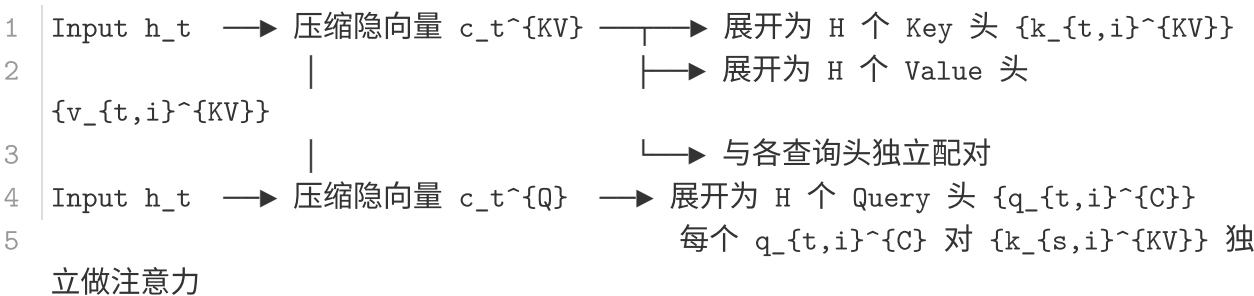
在 128K 上下文下， $k = 2048$ 意味着主注意力仅需计算约 1.6% 的 token 对。索引器虽仍需 $O(L^2)$ 扫描，但其计算量仅为标准 MLA 注意力的一小部分。

2.2 基于 MLA 的 DSA 实例化

2.2.1 MLA 的 MHA 与 MQA 模式

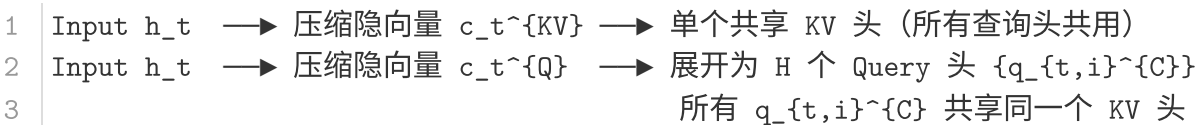
MLA (Multi-head Latent Attention) 通过压缩隐向量来减少 KV 缓存的存储量。在 DeepSeek-V3 系列中, MLA 支持两种模式:

MHA 模式 (训练 & 长序列 Prefilling):



MHA 模式为每个查询头提供独立的键值表示, 表达能力最强。

MQA 模式 (短序列 Decoding & V3.2 DSA):



MQA 模式的 KV 存储量仅为 MHA 的 $1/H$, 且在 kernel 层面对 batch 操作更友好——所有查询头可以同时被 broadcast 到同一组 KV。

2.2.2 为什么 DSA 必须在 MQA 模式上实现

DSA 的稀疏选择——为查询 token 选出 k 个 KV 项——在 MHA 模式下不可行: 不同查询头需要不同的 k 个 KV 项 (因为它们有独立的键投影), 导致 kernel 实现上每个查询头需要独立的 gather/scatter 操作, 极不高效。

在 MQA 模式下, 所有查询头共享同一组 KV 项——DSA 只需一次 **Top-k 选择**, 然后将所有查询头 broadcast 到选中的 k 个 KV 上。这在 CUDA kernel 层面可被高效实现——选中的 KV 项连续排列, 查询头以 batch 维度并行读取。

2.3 继续预训练: 两阶段注入稀疏性

DSA 并非从头训练获得, 而是通过继续预训练从 V3.1-Terminus (已有密集注意力的 128K 上下文模型) 注入稀疏性。训练数据分布与 V3.1-Terminus 的 128K 长上下文扩展数据完全对齐——这意味着没有引入新的数据分布偏移。

2.3.1 阶段一：密集预热——教会索引器"看什么"

参数	值	设计理由
学习率	10^{-3}	较高——索引器参数随机初始化，需要快速收敛
训练步数	1000	仅需少量步数——索引器任务相对简单（模仿已有注意力分布）
每步 128K 序列数	16	$16 \times 128K \times 1000 = 2.1B$
总训练量	2.1B tokens	极低成本（约占主模型预训练的 0.01%）

核心操作：

1. **冻结所有模型参数**——仅 Lightning Indexer 可训练。这确保主模型的行为完全不变，索引器是在"学习模仿"而非"改变分布"。
2. **保持密集注意力**——模型仍然对每对 token 计算完整的注意力分数。
3. **构建目标分布**：对于查询 token t ，将所有 H 个主注意力头的得分沿头维度求和（不同头的关注模式通常互补），然后沿序列维度 L1 归一化：

$$p_{t,s} = \frac{\sum_{i=1}^H \text{AttnScore}_{t,s}^{(i)}}{\sum_{s'=1}^t \sum_{i=1}^H \text{AttnScore}_{t,s'}^{(i)}}$$

4. **KL 散度损失**：

$$\mathcal{L}^I = \sum_t D_{KL}(p_{t,:} \parallel \text{Softmax}(\mathbf{I}_{t,:}))$$

这里对索引分数 $\mathbf{I}_{t,:}$ 施加 softmax 是为了与目标分布 $p_{t,:}$ 对齐（两者都归一化到概率单纯形），使 KL 散度可以自然比较。需要注意的是，训练后的索引器在推理时并不使用 softmax——直接用原始 ReLU 分数做 Top-k。

效果：这个极短的预热阶段教会索引器一个关键的启发式——"在密集注意力中，哪些前序 token 对每个查询 token 贡献最大"。这等价于知识蒸馏——从主注意力（教师）到索引器（学生）。

2.3.2 阶段二：稀疏训练——全模型适应稀疏模式

参数	值	设计理由
学习率	7.3×10^{-6}	远低于预热阶段——全模型已预训练，仅需微调适应稀疏性
训练步数	15000	需要足够步数让模型适应新的稀疏注意力分布
每步 128K 序列数	480	$480 \times 128K \times 15000 = 943.7B$
总训练量	943.7B tokens	约为主模型预训练 token 的 6-7%，但仍属可控
Attention top-k	2048	在 128K 上下文下仅覆盖约 1.6% 的 token

核心操作：

1. **全模型参数解冻**——所有参数参与训练，包括 MLA 的压缩投影矩阵、MoE 的专家权重等。

2. 激活稀疏注意力——正式使用 Top-k 选择机制，主注意力仅计算被选中的 k 个 token。
3. 稀疏约束下的索引器损失——KL 损失仅考虑被选中的 token 集 $\mathcal{S}_t$ ：

$$\mathcal{L}^I = \sum_t D_{KL}(p_{t,\mathcal{S}_t} \parallel \text{Softmax}(\mathbf{I}_{t,\mathcal{S}_t}))$$

这是在稀疏约束下的"有所为有所不为"——索引器被要求在有限的 k 个槽位内最大化捕获重要信息。

4. 梯度解耦（Gradient Detachment）——这是训练稳定性保障的关键设计：

- 1

索引器的损失信号流： $\hat{\mathcal{L}}^I \rightarrow$ Indexer 参数（独立优化）
- 2

主模型的损失信号流： $\mathcal{L}_{LM} \rightarrow$ 所有参数（含 Indexer？不！）

索引器的输入在传递给主模型前被 `.detach()` 截断。这意味着：

- 主模型的语言模型损失不会反向传播到索引器——避免了"主模型试图通过操纵索引选择来降低 LM loss"的危险正反馈循环
- 索引器仅从 $\mathcal{L}^I$ 获取训练信号——使其专注于"选择最重要的 KV"而非"降低 LM loss"
- 如果不用 detach，主模型可能会学会利用路由来作弊（例如通过索引器引导注意力到某些"安全但无信息量"的 token），损害泛化能力

2.4 性能并行性校验：稀疏性引入无损害

DSA 引入稀疏性后，团队通过多维度验证来确保模型质量未遭退化：

校验维度	评测方式	结果	含义
标准基准	知识、推理、代码综合评测	V3.2-Exp $\approx$ V3.1-Terminus	核心能力无退化
人类偏好	ChatbotArena Elo（相同后训练，2025.11.10）	两者 Elo 接近	用户体验无劣化
AA-LCR 长上下文推理	独立第三方长上下文评测	V3.2 +4 分	长上下文反而提升
Fiction.liveBench	多项指标的长上下文小说理解	V3.2 一致优于 V3.1	稀疏注意力可能带来正则化效应

为什么长上下文反而有提升？

这是一个反直觉但重要的发现。一种可能的解释是：密集注意力中，大量低相关度的 KV 对引入了注意力噪声——softmax 的长尾分布使模型被迫在无关 token 上分配注意力"预算"。DSA 的 Top-k 选择起到了硬注意力正则化的作用——迫使模型聚焦于真正重要的 token，反而提升了长上下文理解质量（类似于 dropout 对密集层的正则化效果）。

2.5 推理成本：从理论加速到实际节省

在 H800 GPU 集群部署时（按 \$2/GPU-hour 计价），实测每百万 token 成本：

Prefilling（预填充）阶段：

在短序列（< 约 16K token）时，DSA 的索引扫描开销相对于密集注意力无优势。因此 V3.2 在 kernel 层面实现了 **Masked MHA 模式**——在短序列预填充时，DSA 被降级为"带 mask 的密集 MHA"：

- 密集计算所有 token 对的注意力分数
- 通过 mask 将非 Top-k 的分数置为 $-\infty$ （等效于仅计算 Top-k）
- Masked MHA 比 gather-scatter 的 DSA 在短序列上有更好的 SM 利用率

在长序列时，纯 DSA 的稀疏选择优势逐渐显现——索引器扫描虽为 $O(L^2)$ ，但其 head 数和维度远小于主注意力。

Decoding（自回归解码）阶段：

这是 DSA 效率优势最大的场景：

- V3.1 的解码需要为每个新生成 token 计算与所有历史 KV（最高 128K）的注意力
- V3.2 仅需计算与 $k = 2048$ 个选中 KV 的注意力
- 同时，V3.2 的 KV 缓存也变为稀疏存储——仅需保存被选中的 KV 项

实际成本曲线（论文 Figure 3）：

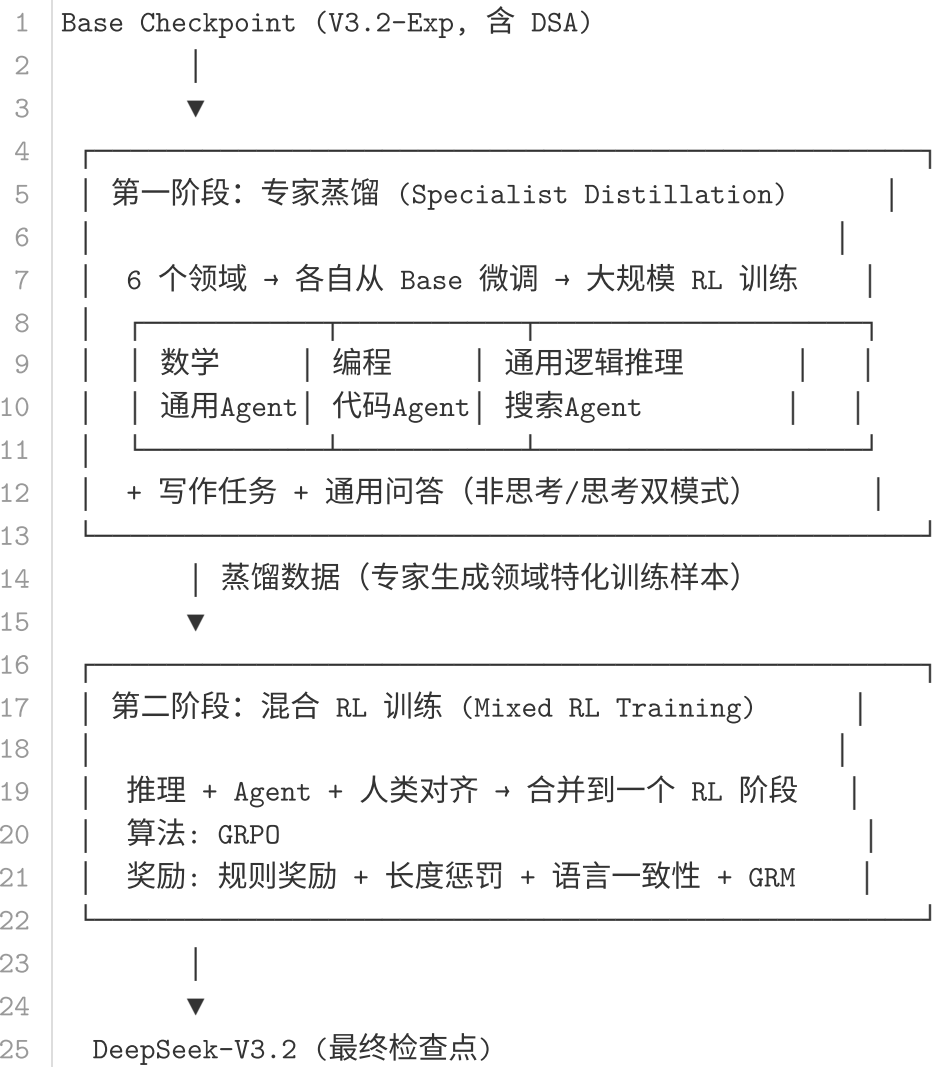
Token 位置	V3.1 每百万 token 成本	V3.2 每百万 token 成本	节省
Prefill 32K	~\$0.50	~\$0.45	~10%
Prefill 128K	~\$0.65	~\$0.55	~15%
Decode 32K	~\$0.30	~\$0.15	~50%
Decode 128K	~\$2.40	~\$0.35	~85%

解码阶段的节省最为显著——因为在长序列解码中，密集注意力的 KV 访问成为主要瓶颈，而 DSA 将每次解码的 KV 访问量从 $O(L)$ （全部历史 token）压缩到了 $O(k)$ （仅 2048 个选中的 token）。

3. 后训练：扩展 RL 与 Agent 合成

V3.2 的后训练是一个大规模的工程系统——后训练计算预算**超过预训练成本的 10%**，远超出开源模型的常规投入。本章详细分析从算法到系统的完整方案。

3.1 后训练流水线



3.1.1 专家蒸馏的动机与设计

为什么需要专家蒸馏而非直接对 Base 模型做多任务 RL?

动机：后训练计算预算有限（尽管已很大），同时优化所有领域会导致目标冲突——数学推理需要长链式思考（long CoT），Agent 任务需要频繁工具调用，通用对话需要简洁直接。一次性 RL 会将这些不同行为模式混入同一个模型，导致各领域都"还行但不精"。

方案：

1. 每个领域**独立训练**专家模型——从相同的 V3.2 Base 出发，分别经历领域特化的 SFT 和 RL
2. 专家模型生成**蒸馏数据**——针对最终检查点所需的全部任务，用对应专家生成高质量的训练样本
3. 蒸馏数据上训练的模型性能**仅略低于**专家，后续的一阶段混合 RL 可以完全抹平差距

这种两阶段设计在计算上是最优的——将"领域特化"的计算量分摊到专家训练（各专家独立训练可高度并行），而"能力融合"仅需一次轻量的混合 RL。

3.1.2 混合 RL 的奖励结构

V3.2 的混合 RL 使用多层次的奖励信号：

奖励类型	适用任务	形式
规则式结果奖励	推理、Agent（可验证）	二元或连续值（答案对错、测试通过率）
长度惩罚	所有思考模式	对输出 token 数施加代价，限制冗长
语言一致性奖励	所有任务	防止 RL 训练中的语言漂移（模型逐渐偏向某语言输出）
生成式奖励模型（GRM）	通用对话、难验证任务	每个 prompt 有专属的评估 rubric，GRM 基于 rubric 打分

GRM 的设计尤为关键——它不是训练一个独立的标量奖励模型（如 RLHF 的传统做法），而是利用 **Actor 网络本身作为 GRM**，在评判（judging）和生成（generating）能力上联合优化。这种方式使得模型的内部推理能力自然融入评估过程，仅需极少量的多样化人工标注即可泛化。

3.2 扩展 GRPO：从理论基础到四项稳定训练技术

3.2.1 GRPO 完整目标函数逐项解析

GRPO 的优化目标（论文 Equation 5）：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{old}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) - \beta D_{KL} \right]$$

逐项分解：

(a) **组采样（Group Sampling）**：对每个问题 q 从旧策略 π_{old} 采样 G 个响应 $\{o_1, \dots, o_G\}$ 。 G 通常取 4-16——足够构建统计显著的组内比较，又不至于浪费过多推理预算。

(b) **重要性采样比**：

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{old}(o_{i,t}|q, o_{i,<t})}$$

衡量当前策略 π_{θ} 与生成数据的旧策略 π_{old} 在 token 级别的分歧度。 $r_{i,t} > 1$ 意味着当前策略认为该 token 比旧策略更可能生成——这是一个“好 token”的信号（在正 advantage 下）。

(c) **组归一化 Advantage**：

$$\hat{A}_{i,t} = R_i - \text{mean}(\{R_1, \dots, R_G\})$$

这是 GRPO 的核心创新——advantage 基于**组内比较**而非全局比较。好处：无需维护 value network（如 PPO 需要），消除 value function 估计误差；组内归一化自动适应不同问题的难度差异（难题的奖励整体低，但组内相对优劣仍可区分）。

(d) **PPO-style Clipping**: $\min(r\hat{A}, \text{clip}(r, 1 - \varepsilon, 1 + \varepsilon)\hat{A})$ 防止单步策略更新过大——当 $r_{i,t}$ 偏离 1 太多时，梯度被截断。典型 $\varepsilon = 0.2$ 。

(e) **KL 惩罚**: $-\beta D_{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ 防止策略过度偏离参考策略 π_{ref} (通常是 SFT 模型)。 β 控制惩罚强度。

3.2.2 无偏 KL 估计：为何重要及如何实现

K3 估计器的偏差问题：

传统的 K3 估计器定义为：

$$D_{KL}^{K3} = \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_{\text{old}}(o_{i,t})} \cdot \log \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_{\text{old}}(o_{i,t})}$$

问题在于：当 $\pi_\theta(o_{i,t}) \ll \pi_{\text{ref}}(o_{i,t})$ (当前策略远低于参考策略时)，K3 的梯度 $\nabla_\theta D_{KL}^{K3}$ 在期望上不等于真实的 KL 梯度。具体来说，K3 对这些"当前策略不认可但参考策略认可的 token"分配了过大的梯度权重，在随后的迭代中这些噪声梯度累积，导致采样质量退化。

V3.2 的无偏修正：

通过引入当前策略 π_θ 与旧策略 π_{old} 的重要性采样比 ($r_{i,t}(\theta)$)，推导出无偏 KL 估计：

$$D_{KL}^{\text{unbiased}}(\pi_\theta(o_{i,t}) \parallel \pi_{\text{ref}}(o_{i,t})) = r_{i,t}(\theta) \cdot \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_\theta(o_{i,t})} - \log \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_\theta(o_{i,t})} - 1$$

本质上，这是在 K3 估计器上应用了重要性采样矫正。项 -1 来自 KL 散度定义中的常数偏移 ($\sum_x p(x) \log q(x) - \sum_x p(x) \log p(x)$ 中 $\sum_x p(x) \log p(x)$ 在 $p = \pi_\theta$ 时的展开)。

β 的领域自适应

领域	KL 惩罚强度	原因
数学推理	极弱 / 零	数学推理是冷启动后的"探索性"过程，宽松的 KL 允许模型在长 CoT 空间中自由探索
代码生成	中等	代码语法约束和特定模式需要保持 (偏离太远会产生语法错误)
通用对话	较强	需要保持安全对齐和语言风格，防止 RL 导致行为模式剧烈变化

3.2.3 Off-Policy 序列掩码：选择性学习

Off-policy 的双重来源：

在标准的 RL 训练管线中：

- 推理框架生成一批 rollout $\rightarrow$ 拆分为多个 mini-batch $\rightarrow$ 多次梯度更新 (数据重用引入 off-policy)

2. 推理和训练框架可能因实现细节差异（如 MLA 的 MHA/MQA 模式切换、FP8 精度处理、attention kernel 差异）导致同一输入产生略微不同的 logits $\rightarrow$ 采样策略 π_{old} 与训练中计算的 π_{old} 有偏差

V3.2 将这两种来源统一处理—— π_{old} 取**推理框架**实际返回的采样概率（而非训练框架重新计算的概率），因此两个来源的 off-policy 程度都被 KL 散度 $\frac{1}{|O_i|} \sum_t \log \frac{\pi_{\text{old}}(O_{i,t})}{\pi_{\theta}(O_{i,t})}$ 捕获。

掩码规则与直觉：

$$M_{i,t} = \begin{cases} 0 & \hat{A}_{i,t} < 0 \text{ 且 } \frac{1}{|O_i|} \sum_{t=1}^{|O_i|} \log \frac{\pi_{\text{old}}(O_{i,t})}{\pi_{\theta}(O_{i,t})} > \delta \\ 1 & \text{otherwise} \end{cases}$$

只在两个条件**同时**满足时掩码：

1. **负 advantage**：该响应比组内平均水平差
2. **高 off-policy 度**：采样时的策略与当前策略分歧过大

为什么只掩码负 advantage 序列而不掩码正 advantage 序列？

核心直觉："从成功中学习"和"从失败中学习"的信息价值不对称：

- 正 advantage 样本：即使有点 off-policy（旧策略的偶然好样本），模型学习"这类行为方向大致正确"仍有益——不会破坏已学到的能力
- 负 advantage 样本：高度 off-policy 的负样本可能来自旧策略的**偶然坏输出**——当前策略本来不会生成，学习避免它反而可能引入错误信号，甚至破坏优化稳定性

这本质上是"宁可漏过一个负例，不可学错一个负例"的保守策略。

损失函数中的掩码应用：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|O_i|} \sum_{t=1}^{|O_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t} \right) \cdot M_{i,t} - \beta D_{KL} \right]$$

掩码 $M_{i,t}$ 乘在 PPO 项上——直接将该序列从策略梯度更新中移除。KL 项不受掩码影响（KL 是对整个策略的约束，与具体序列无关）。

3.2.4 Keep Routing：MoE 的 RL 稳定性保障

问题根源：

MoE 模型推理时，每个 token 通过路由网络（通常是 softmax over expert logits + top-k）选择专家。在 RL 上下文中：

1. 推理框架和训练框架可能因 kernel 实现差异产生略微不同的路由 logits
2. 即使 logits 相同，策略更新（ $\pi_{\text{old}} \rightarrow \pi_{\theta}$ ）会改变路由网络的权重
3. 同一 token 在推理和训练时可能被路由到**不同的专家**

后果：训练时优化的专家参数与推理时实际被激活的专家不匹配——等价于在一个不一致的参数空间上做梯度下降，导致训练震荡甚至发散。

解决方案：

- 推理框架采样时，记录每个 token 的**实际路由决策**（哪些专家被激活）
- 训练时，**强制使用相同的路由决策**——即使当前路由网络的输出不同，也按记录的路由执行
- 梯度仅更新被记录路由激活的专家参数

这项工作自 DeepSeek-V3-0324 起已成为 V3 系列的标准训练实践，V3.2 延续了这一设计。

3.2.5 Keep Sampling Mask：动作空间一致性

问题：

Top-p（nucleus）采样通过截断低概率尾部分布来提升生成质量。但在 RL 中：

- π_{old} 采样时的动作空间 = {被 Top-p 保留的 token}
- π_{θ} 训练时的动作空间 = {词表中全部 token}

重要性采样比 $r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t})}{\pi_{\text{old}}(o_{i,t})}$ 要求两个分布定义在**相同**的动作空间上。如果 π_{old} 将某些 token 的概率截断为零（因为被 Top-p 过滤），但 π_{θ} 给这些 token 非零概率—— $r_{i,t}$ 的分母为零，比值无定义。

解决方案：

- 采样时记录 Top-p/Top-k 的**截断掩码**（哪些 token 被过滤掉）
- 训练时将同样的掩码应用于 π_{θ} ——将过滤掉的 token 概率置零并重新归一化
- 确保 $r_{i,t}$ 的计算在相同动作子空间上进行

实践价值：top-p 采样 + Keep Sampling Mask 的组合在 RL 训练中被验证为有效保持**语言一致性**——防止模型在 RL 过程中逐渐偏向某种语言输出模式（如从中文漂移到英文）。

3.3 工具调用中的思考（Thinking in Tool-Use）

工具调用 + 推理思考的结合是 V3.2 最具特色的后训练创新。本节深入分析这一结合面临的挑战和解决方案。

3.3.1 上下文管理的两种模式

DeepSeek-R1 的原始策略：第二回合消息（无论是 User 还是 Tool）到达时全部丢弃推理内容。简单直接，但在多轮工具调用场景中导致灾难性的 token 浪费——每轮工具调用后模型都需要从头重新推理完整问题。

V3.2 的条件保留策略：

1	
2	工具调用场景的思考内容管理
3	
4	Round 1: User Question
5	→ <think>推理路径1</think>
6	→ Tool Call 1
7	→ Tool Output 1
8	
9	Round 2: 新消息是 Tool Output 1
10	→ 保留 <think>推理路径1</think> ← 关键!
11	→ <think>基于Tool Output1继续推理</think>
12	→ Tool Call 2
13	→ Tool Output 2
14	
15	Round 3: 新消息是 Tool Output 2
16	→ 保留所有历史推理（路径1+路径2）
17	→ <think>基于累计信息推理</think>
18	→ Final Answer
19	
20	新 User 消息到达时:
21	→ 丢弃所有 <think> 内容
22	→ 保留工具调用历史和结果
23	

设计哲学：User 消息代表"任务切换"（新问题），应清空推理上下文；Tool 消息代表"任务延续"（更多信息），应保持推理连贯性。

兼容性注意事项：部分 Agent 框架（Roo Code、Terminus）通过模拟用户消息来传递工具输出——在这些框架下，V3.2 的条件保留策略不会触发（因为框架发送的是 User 消息，推理内容被丢弃）。论文明确建议此类框架使用**非思考模式**以获得最佳性能。

3.3.2 Cold-Start：用提示工程启动思考+工具融合

数据现状：团队拥有两类独立的训练数据：

- 推理数据：有 <think> 标签的 Long CoT 推理，但无工具调用
- Agent 数据：有工具调用的多轮对话，但无 <think> 推理

Cold-Start 方案：通过精心设计的系统提示使模型自发融合两者。

关键设计（论文 Appendix B, Table 8 的完整提示模板）：

```
1 You are a helpful assistant with access to a Python interpreter.
2
3 - You may use the Python tool **multiple times** during your reasoning,
4   a.k.a in <think></think>, with a maximum of 20 code executions.
5
6 - Call the Python tool early in your reasoning to aid in solving the
7   task. Continue reasoning and invoking tools as needed until you reach
8   the final answer. Once you have the answer, stop reasoning and present
9   your solution using Markdown and LaTeX.
10
11 - Do NOT invoke any tools in your presented final solution steps.
12
13 - To improve efficiency and accuracy, you should prefer code execution
14   over language-based reasoning whenever possible. Keep your reasoning
15   succinct; let the code do the heavy lifting.
```

这个提示精妙地实现了几个关键约束：

- 1. 工具调用在 `<think>` 内部：工具调用是推理过程的一部分，不是推理之后的操作
- 2. 最多 20 次代码执行：防止陷入无限循环，同时给予足够的探索空间
- 3. 最终答案不使用工具：推理阶段用工具探索，最终方案是纯粹的文本输出
- 4. 偏好代码执行而非语言推理：引导模型在推理中实际运行代码验证想法

效果与局限：Cold-Start 让模型偶尔生成理想的"思考+工具"轨迹，为后续 RL 提供了种子数据。但初始质量不稳定——有些轨迹中工具调用时机不当，有些推理与工具结果脱节。

3.3.3 大规模 Agent 任务合成：从数据到能力的完整链路

V3.2 的 Agent 任务合成覆盖四类场景，总计 85,267 个任务，1,827+ 个独特环境：

Agent 类型	任务数	环境真实度	Prompt 来源	核心技术挑战
Code Agent	24,667	真实 GitHub	抽取	自动构建可复现环境
Search Agent	50,275	真实搜索 API	全合成	多 Agent 数据质量控制
General Agent	4,417	全合成	全合成	难度递增与自动验证
Code Interpreter	5,908	真实 Jupyter	抽取	跨语言测试解析

Search Agent：互联网搜索的规模化训练数据

Search Agent 的训练数据规模最大（50,275 个任务），且完全由合成生成。

合成的结构性动机：真实用户的搜索查询存在隐私问题，且分布集中在流行话题——缺少"长尾"查询。合成管线通过主动从 web 语料中挖掘信息丰富的长尾实体来弥补这一缺陷。

多 Agent 流水线详细过程：

```
1 Phase 1: 长尾实体采样与探索
2   └─ 从大规模 web 语料中采样信息丰富但低频的实体
3     │   (如：特定历史事件、罕见科学概念、小众技术产品)
4     ▼
5 Phase 2: 问题构造 (Question Construction Agent)
6   └─ 对每个实体，Agent 使用搜索工具进行多深度/广度探索
7     │   - 可配置深度参数 (递归搜索几层链接)
8     │   - 可配置广度参数 (每层打开多少链接)
9     │   → 整合发现的信息 → 生成问答对
10    ▼
11 Phase 3: 多答案生成 (Multi-Answer Generation)
12   └─ 多个异质 Agent 并行生成候选答案
13     │   - 不同检查点 (同一模型的不同 RL 训练阶段)
14     │   - 不同系统提示 (不同的搜索策略引导)
15     │   - 不同采样参数 (temperature, top-p)
16     │   目的：保证答案的多样性，增加"所有候选错误"的概率
17     ▼
18 Phase 4: 答案验证 (Verification Agent)
19   └─ 带搜索能力的验证 Agent 进行多轮验证
20     │   - 逐一验证 ground-truth 答案是否正确
21     │   - 逐一验证所有候选答案是否确实错误
22     │   → 仅保留：ground-truth 正确 + 所有候选错误
23     │   → 若 ground-truth 错误或任一候选正确 → 丢弃整个样本
24     ▼
25 Phase 5: 数据增强
26   └─ 从已有的 helpful RL 数据集中筛选搜索工具可增值的实例
27     │   补充合成数据难以覆盖的日常使用场景
28     ▼
29 Phase 6: 多维度评分
30   └─ 开发评估 rubric (事实准确性、搜索效率、回答完整性...)
31     └─ 生成式奖励模型基于 rubric 评分 → 驱动 RL 优化
```

质量保证的核心机制：Phase 4 的"正确答案 + 全部候选错误"双重过滤保证了每个保留样本的可学习性——如果至少一个候选能答对，说明该问题对当前模型能力而言是"可解的"，RL 才有优化的空间。

Code Agent：从 GitHub 到可复现测试环境

代码 Agent 的训练环境是真实的——从 GitHub 挖掘 issue-PR 对并构建可测试环境。

从挖掘到验证的完整流水线：

```
1 Phase 1: Mining (挖掘)
2   └─ 从 GitHub 挖掘数百万 issue-PR 对
3     └─ 覆盖语言: Python, Java, JS, TS, C, C++, Go, PHP
4   ▼
5 Phase 2: Filtering (过滤)
6   └─ 启发式规则:
7     │   • Issue 描述长度 > 合理阈值 (过短 = 信息不足)
8     │   • PR 修改文件数在合理范围 (过大 = 重构非修复)
9     │   • Issue 和 PR 的时间相关性 (关联强度)
10  └─ LLM 判断:
11     │   • Issue 是否包含可操作的 bug 描述
12     │   • PR 是否是针对该 issue 的修复
13     │   • 是否存在配套的测试修改
14     └─ 要求: Issue 合理 + PR 相关 + Gold Patch + Test Patch
15   ▼
16 Phase 3: Environment Setup (环境设置)
17   └─ DeepSeek-V3.2 驱动自动环境设置 Agent:
18     │   ① 分析仓库的依赖声明 (requirements.txt, pom.xml, go.mod...)
19     │   ② 安装所有依赖包 (处理版本冲突和兼容性问题)
20     │   ③ 执行测试套件 → 验证初始状态
21     │   ④ 应用 Gold Patch → 重新执行测试
22     │   ⑤ 验证结果:
23     │       • F2P (False-to-Positive): Gold Patch 修复后通过数 > 0
24     │       代表 patch 确实修复了至少一个之前失败的测试
25     │       • P2F (Pass-to-Fail): Gold Patch 后失败数 = 0
26     │       代表 patch 没有引入新的回归
27     └─ → 仅 F2P>0 且 P2F=0 时判定环境构建成功
28   ▼
29 Phase 4: Output (输出)
30   └─ 最终获得数万个可复现的 issue 修复环境
31     统一 JUnit 格式测试输出, 跨语言一致解析
```

F2P/P2F 指标的设计精妙之处：

- F2P > 0 保证问题是**真实存在**且可以被修复的（排除了"本质不可解"的环境噪声）
- P2F = 0 保证修复是**正确的**
——没有引入回归（排除了"修复导致新问题"的低质量样本）
- 使用 JUnit 标准格式统一跨语言的测试输出——agent 看到的始终是结构化的测试结果，无论后端语言是什么

General Agent：自动合成环境的极致泛化

General Agent 的任务和环境完全合成，规模为 1,827 个独有环境 + 4,417 个任务。这是最具创新性的部分——因为它证明了合成数据可以泛化到真实环境。

自我迭代的合成工作流：

```
1  [自动合成Agent工作流]
2  |
3  Input: 任务类别 (如"旅行规划")、Sandbox (bash + search 工具)
4  |
5  └─ Step 1: 数据准备
6  |   Agent 使用 bash/search 工具 → 生成/检索相关数据 → 存入 sandbox DB
7  |   例如: 爬取真实城市、酒店、景点、餐厅数据
8  |
9  └─ Step 2: 工具集成
10 |   Agent 合成任务特定的工具函数 (每个工具是一个 function)
11 |
12 |   例如 trip planning 的 15 个工具:
13 |   • get_all_cities(), get_all_hotels_by_city(city)
14 |   • get_all_restaurants_by_city(city)
15 |   • get_all_attractions_by_city(city)
16 |   • get_city_by_hotel(hotel) ← 反向查询
17 |   • get_weather_by_city_date(city, date)
18 |   • get_inter_city_transport(from, to)
19 |   • get_infos_by_hotel(info_keywords, hotel) ← 获取详细信息
20 |   • submit_result(answer_text) ← 提交答案
21 |
22 └─ Step 3: 迭代难度递增 + 自动验证
23 |   ┌─ 初始: 简单任务 + solution + verifier
24 |   |   约束: solution 仅能调用工具函数/逻辑计算
25 |   |       不能调用其他函数或直接访问 DB
26 |   |       → 确保任务只能通过工具接口解决
27 |   |   验证: verifier 验证 solution 输出
28 |   |       → 不通过则修改 solution/verifier 直到通过
29 |   |
30 |   └─ 迭代: 增加难度
31 |       例如 trip planning:
32 |       简单 → 找三个城市不重复的酒店
33 |       中等 → 加上预算约束
34 |       困难 → 加上天气、交通、评分多约束 + 条件分支
35 |       (见 paper Table 中的完整 trip planning 例子)
36 |
37 ┌─ 按需增强工具集:
38 |   如果当前工具不足以求解, Agent 自动添加新工具
```

39
40
41
42

|
└─ Step 4: RL 过滤
 对合成任务做 RL → 仅保留 pass@100 > 0 的实例
 → 最终 1,827 个环境 / 4,417 个任务

设计精髓：

- 1. **Solution 受限访问**：solution 不能直接访问数据库，只能通过工具接口——这强制了 agent 行为与人类解决类似问题的模式一致。这种约束使合成环境的行为特征**逼近真实环境**。
- 2. **Verifier 自动验证**：verifier 是一个纯 Python 函数——输入 solution 输出、判断正确性。这使得大规模自动验证成为可能（无需人工标注），且验证逻辑透明可审计。
- 3. **迭代难度递增**：通过 Agent 自我评估 + 按需增强工具——避免了"一次性合成大量难以求解的垃圾任务"的陷阱。只有工具集充足的困难任务才被保留。
- 4. **RL 过滤 (pass@100 > 0)**：仅保留至少有一次（在 100 次尝试中）被成功求解的任务——确保 RL 训练中有正向奖励信号。

合成数据的泛化验证——关键实验：

训练方案	Tau2Bench	MCP-Mark	MCP-Universe
V3.2-SFT（无 RL）	baseline	baseline	baseline
+ 仅在代码/搜索环境 RL（无合成）	无提升	无提升	无提升
+ 仅在合成数据上 RL	显著提升	显著提升	显著提升
DeepSeek-V3.2（最终）	80.3	38.0	45.9

在 MCP-Mark 和 MCP-Universe 上的泛化增益尤为显著——因为这两个 benchmark 的**具体环境和工具在训练中从未出现**。仅在合成数据上 RL 就能带来提升，而仅在代码/搜索环境 RL 则无此效果——这证明了合成数据的**跨环境泛化能力**是其独特价值所在，而非代码/搜索 RL 的附带收益。

4. 评测结果

4.1 主模型性能

推理能力

Benchmark	DS-V3.2-Thinking	GPT-5-High	Gemini-3.0-Pro
AIME 2025	93.1	94.6	95.0
HMMT Feb 2025	92.5	88.3	97.5

Benchmark	DS-V3.2-Thinking	GPT-5-High	Gemini-3.0-Pro
HMMT Nov 2025	90.2	89.2	93.3
IMOAnswerBench	78.3	76.0	83.3
HLE (text)	25.1	26.3	37.7
LiveCodeBench	83.3	84.5	90.7
Codeforces Rating	2386	2537	2708

Agent 能力

Benchmark	DS-V3.2-Thinking	GPT-5-High	Gemini-3.0-Pro
Terminal Bench 2.0	46.4	35.2	54.2
SWE Verified	73.1	74.9	76.2
SWE Multilingual	70.2	55.3	—
BrowseComp	67.6*	54.9	45.8
BrowseCompZh	65.0	63.0	—
Tool-Decathlon	35.2	29.0	36.4
MCP-Universe	45.9	47.9	50.7
MCP-Mark	38.0	50.9	43.1
τ^2 -Bench	80.3	80.2	85.4

*：使用 Discard-all 上下文管理策略

关键发现：

- V3.2 在推理任务上与 GPT-5 持平，在数学领域显著超过
- 在 Agent 领域大幅缩小开源与闭源模型的差距
- 在 MCP 和 Tool-Decathlon 等 Tool-Use 基准上显著超过所有其他开源模型
- 不足：超长推理轨迹常超过 128K 上下文限制（MCP-Mark GitHub/Playwright 任务中），影响最终成绩

Token 效率对比

Benchmark	DS-V3.2	Kimi-K2	GPT-5	Gemini-3.0
AIME 2025	93.1 (16k)	94.5 (24k)	94.6 (13k)	95.0 (15k)
HMMT Feb	92.5 (19k)	89.4 (31k)	88.3 (16k)	97.5 (16k)
Codeforces	2386 (42k)	—	2537 (29k)	2708 (22k)

DS-V3.2 以显著少于 **Kimi-K2** 的 token 量达到可比性能。但与 Gemini-3.0-Pro 相比仍有 token 效率差距。

4.2 DeepSeek-V3.2-Speciale

这是推至极限的推理变体：仅推理数据训练，降低长度惩罚。

Benchmark	DS-V3.2-Thinking	DS-V3.2-Speciale	Gemini-3.0-Pro
AIME 2025	93.1	96.0	95.0
HMMT Feb 2025	92.5	99.2	97.5
HMMT Nov 2025	90.2	94.4	93.3
IMOAnswerBench	78.3	84.5	83.3
LiveCodeBench	83.3	88.7	90.7
Codeforces Rating	2386	2701	2708
HLE (text)	25.1	30.6	37.7

顶级竞赛成绩（通用模型，无针对性训练）

竞赛	分数	奖牌	备注
IMO 2025	35/42	金牌	结合 DeepSeekMath-V2 技术
CMO 2025	102/126	金牌	英文版, generate-verify-refine
IOI 2025	492/600	金牌	排名第 10, 多阶段过滤
ICPC WF 2025	10/12	金牌	排名第 2, 每题最多 32 候选

IOI 评测策略：每题采样 500 个候选解 → 过滤无效/拒绝的 → 选最长 50 个的思考轨迹 → 提交（每 50 次提交）

但这带来了 token 效率代价：输出 token 数远高于 Gemini-3.0-Pro（AIME 23k vs 15k, HMMT 27k vs 16k）。

4.3 搜索 Agent 的上下文管理

搜索 Agent 工作流常超出 128K 限制。四种上下文扩展策略：

策略	原理	BrowseComp 得分	平均步数
Summary	总结溢出轨迹重新启动	60.2	364
Discard-75%	丢弃前 75% 工具调用历史	62.3	—
Discard-all	丢弃全部工具调用历史	67.6	—
Parallel-fewest-step	并行 N 条轨迹选最少步	67.5	—
Baseline (无管理)	—	51.4	140

结论：测试时计算可通过串行（上下文管理）或并行（多轨迹采样）两种方式扩展。Discard-all 虽简单，但在效率和扩展性上表现都很好。

5. 工程实践亮点

5.1 DSA 的工程实现

- 基于 MLA 的 MQA 模式实现——使稀疏 KV 可跨查询共享，利于 kernel 层面高效实现
- 短序列 prefilling 使用 **Masked MHA 模式**模拟 DSA——在短上下文获得更高效率
- Lightning Indexer 使用 **FP8** 精度实现，小头数设计极致轻量
- 开源实现随 V3.2-Exp 发布

5.2 RL 基础设施

- **批推理 + 多 mini-batch 训练**：高效利用 GPU，但在训练-推理一致性上需要额外技术（Keep Routing, Keep Sampling Mask）
- 推理和训练框架的实现差异被显式处理（通过 Off-Policy Sequence Masking 吸收两方面的 off-policy 程度）
- 无偏 KL 估计器使得不同领域可独立调整 KL 惩罚强度

5.3 Agent 合成基础设施

- 多 Agent 流水线（问题构造 → 答案生成 → 验证）完全基于 DeepSeek-V3.2 自身
- 代码 Agent 环境支持 8 种编程语言，JUnit 标准统一测试解析
- General Agent 环境的**自动化难度递增**机制——Agent 自我评估工具是否足够，不足则增强

5.4 对齐技术

- **生成式奖励模型**：针对通用任务，每个 prompt 有专属的评估 rubric
- **语言一致性奖励**：防止 RL 训练中的语言漂移
- **长度惩罚**：平衡推理深度与 token 成本

6. 局限性与未来方向

6.1 世界知识差距

由于总训练 FLOPs 少于前沿闭源模型，V3.2 的世界知识广度仍落后于 Gemini-3.0-Pro。解决方案：扩大预训练计算规模。

6.2 Token 效率不足

V3.2 通常需要更长的生成轨迹才能匹配 Gemini-3.0-Pro 的输出质量。Special 变体更是严重牺牲效率换取性能。未来方向：**优化推理链的智能密度**。

6.3 复杂任务求解

在最困难任务上的表现仍落后于前沿模型。需要进一步改进基础模型和后训练配方。

6.4 上下文长度限制

128K 的上下文窗口在 Agent 工作流中频繁不足（MCP-Mark 中约 20% 测试用例超限）。上下文管理策略是临时方案，更长上下文支持是必要方向。

参考文献: DeepSeek-AI. "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models." 2025.

报告生成日期: 2026-04-26